

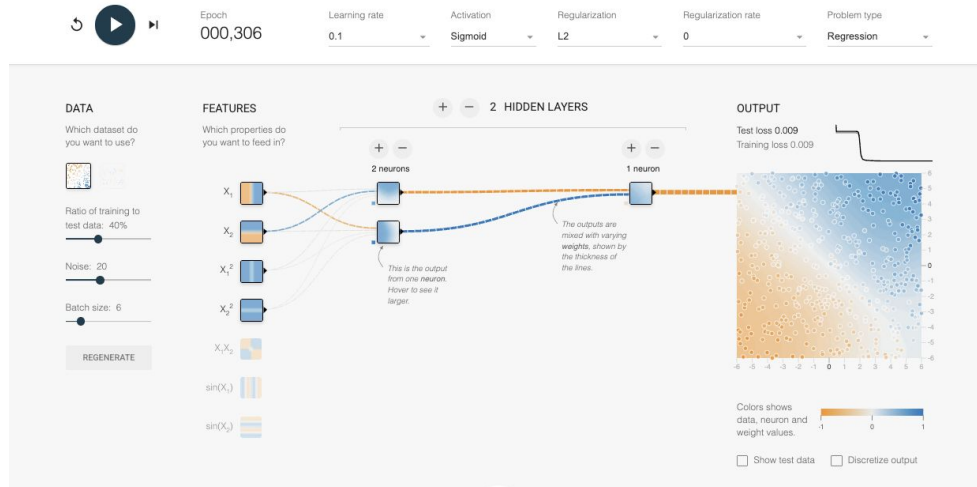


Module 3

Why Interactivity Accelerates AI Discovery

We touched on this earlier, but interactivity deserves its own module because it genuinely changes how you work. Imagine your data, models, visualizations and user input all working together in a single system that updates automatically. This section shows you what that unified system looks like in practice and how it will transform the way you work.

Tinker With a **Neural Network** Right Here in Your Browser.
Don't Worry, You Can't Break It. We Promise.



To sort of give you an idea,

You might remember TensorFlow Playground — it's been around for years, but it's still one of the clearest examples of why interactivity matters.

You pick a dataset, add a layer, change the activation, tweak a feature — and the network retrains right in front of you. You just edit, and the model responds.

What makes it powerful isn't the visuals. It's the loop. You form a hypothesis, you change one thing, you see what happens — and you actually *learn* something about how the model behaves.



Interactive Computation as a Unified System

- Interactivity turns the notebook into one live system.
- Interactive exploration improves model development.
- Visual feedback speeds up debugging.

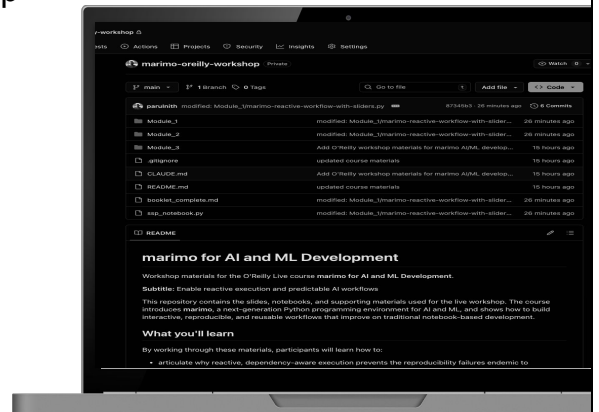
Interactive computation becomes powerful when the notebook behaves like one live graph instead of a stack of disconnected steps. In that kind of workflow, user input is not separate from the computation. Tables, widgets, plots, and model outputs all become part of the same system. The key idea is continuity: exploration does not end when modelling begins. The same notebook lets you move straight from inspecting data to changing model inputs, comparing results, and debugging errors.



Let's See It in Action

github.com/parulnith/marimo-oreilly-workshop

Module_3



THE
QUIZ

Poll: In a notebook, what actually drives the automatic updates when a user changes a UI control like `mo.ui.multiselect(...)`?

- marimo reruns the entire notebook from top to bottom
- Only the cell containing the UI component reruns
- marimo tracks dependencies and recomputes only affected downstream cells
- The user's browser handles all updates without Python execution

Correct answer: C



Poll: What makes the debugging workflow in marimo notebook fundamentally different from a traditional notebook workflow?

- Errors are visualized instead of printed
- Misclassified samples can be interactively selected and immediately analyzed within the same flow
- Debugging is handled automatically by marimo
- The model retrains itself when errors are detected

Correct answer: B

Poll: What is the key difference between `mo.ui.data_editor(df)` and `mo.ui.dataframe(df)` in this workflow?

- Both allow editing, but only one supports sorting
- `data_editor` modifies data that feeds computation, while `dataframe` is for inspection only
- `dataframe` updates downstream cells, but `data_editor` does not
- There is no functional difference between them

Correct answer: B